

BHTTP/1.0 — Binary HTTP

Course project · HTTP, in binary · two tracks, one protocol · now **you** write the spec | Java ref: bserve ./www 9000 · bcurl -v localhost:9000/index.html

1 Transport & framing

TCP, **one persistent connection**. All integers unsigned **big-endian** (network order). A frame = **9-byte header** + exactly **Length** **payload bytes**: readFully(9) then readFully(Length) — partial TCP reads are normal TCP behaviour, not errors.

MAX_FRAME = 16,777,215 ($2^{24}-1$); a larger Length is a framing error → close. Timeouts: server idle 60 s, client connect 5 s / read 10 s. Every multi-byte field below is big-endian.

```
0 1 2 3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| Payload Length (24) |
+-----+-----+-----+-----+-----+-----+-----+
| Type (8) | Flags (8) |R| Request-ID (31) |
+-----+-----+-----+-----+-----+-----+
| Payload (Length bytes) ...
```

Field	Width	Rule / defense (why 24 / 8 / 8 / 31?)
Length	24	Payload-only length. 16-bit (64 KiB) fragments every image and forces chatty fragmentation; 32-bit invites a 4 GiB DoS allocation on one lying header. 24 fits a whole small file / large chunk, bounds receiver memory, and drives unknown-skip : skip = read exactly Length bytes.
Type	8	0x01 REQUEST, 0x02 RESPONSE, 0x03 DATA — 3 used, 253 spare for v2 . 4 bits (16 values) leaves no extension room; 16 bits wastes a byte on every frame and invites type sprawl.
Flags	8	Per-type bits. END_STREAM=0x01, END_HEADERS=0x02. Eight booleans is enough for v1; fewer forces new types per boolean, more encourages flag explosion. Reserved bits MUST be 0 on send, ignored on receive.
R + ID	1+31	Top bit MUST be 0 (keeps Java int non-negative, reserves a future bit a la HTTP/2). ID 0 illegal in v1 (reserved for connection-level frames in v2). Client uses odd 1, 3, 5... , server echoes. Even with sequential keep-alive in v1, IDs catch mismatch and give v2 multiplexing with no header change .

A receiver meeting a frame type it does not know MUST skip it cleanly: read + discard exactly Length bytes, ignore flags/ID, continue — MUST NOT close. That is how we leave room for a version 2.

2 Frame payloads

REQUEST 0x01: method:u8 + pathLen:u16 + path:UTF-8 + numHeaders:u16 + headerBlock. method: 0x01 GET, 0x02 HEAD; any other value → **400**. path **MUST** start /, contain no NUL, be strict UTF-8, ≤2048 B. Flags **MUST** be 0x03 (END_STREAM|END_HEADERS) in v1 — requests carry no body. numHeaders **MUST** equal the entries actually in headerBlock, else 400.

RESPONSE 0x02: status:u16 + numHeaders:u16 + headerBlock, status 100–599. END_STREAM=1 → no body follows; 0 → ≥1 DATA frame follows. **DATA** 0x03: opaque file bytes, exactly Length bytes; END_STREAM=1 on the last chunk, else 0. Zero-length file and HEAD → RESPONSE with END_STREAM=1 and zero DATA frames. A DATA frame before any RESPONSE, or a RESPONSE with a mismatched ID, is a transport error.

3 Headers — HPACK mechanisms 1 + 2 in an evening

Static table — the ten names we actually send: 0x01 :path 0x02 :status (reserved) 0x03 content-type 0x04 content-length 0x05 date 0x06 server 0x07 host 0x08 user-agent 0x09 accept 0x0A connection. Each header entry starts with a one-byte discriminator: 0x01–0x0A → index:u8 + valueLen:u16 + value:UTF-8; 0x00 → literal 0x00 + nameLen:u16 + name:UTF-8 + valueLen:u16 + value:UTF-8 (name **MUST** be lowercase [a-z0-9-]); receiver **MUST** accept and preserve unknown literal names). Any other discriminator byte → **400**. Guards (all in-spec, all 400 on violation): **≤64 headers, name ≤256 B, value ≤8 KiB, whole block ≤64 KiB**. Deliberately no Huffman coding, no dynamic table, no variants in v1 — fixed u16 length prefixes keep an-evening scope while demonstrating the two key HPACK ideas: indexed names + length-prefixed literal fallback.

4 Path → file & status

Order matters: strip #fragment, then strip ?query → percent-decode %XX (bad % → **400**) → / maps to /index.html and a trailing / appends index.html → normalize (/a/./b → /b). Result **MUST** stay under the root (including a no-follow symlink real-path check), else **400** — an escape attempt is an attack, not a 404. Regular file → **200** with content-type + content-length + date (IMF-fixdate, GMT) + server:bserve/1.0, body streamed as DATA in 16 KiB chunks. Missing file or directory without index.html → **404** with a small text/plain "Not Found\n" body (so the DATA path is still exercised). HEAD → same headers with content-length = file size but no DATA. MIME map: html/htm text/html, css

text/css, js text/javascript, json application/json, txt text/plain, png image/png, jpg/jpeg image/jpeg, gif image/gif, svg image/svg+xml, else application/octet-stream. I/O failure mid-response → **500**, then close (length already promised).— client treats any 4xx/5xx as an error exit.

5 Connection lifecycle, CLIs & interop

Server bserve <root> <port>: bind, accept loop, one thread per connection; per-connection loop is read header → read payload → dispatch → RESPONSE [+DATA] → flush → repeat. Semantic errors (bad method / path / headers / traversal) → **4xx RESPONSE and keep the connection open** for the next request. Fatal framing (truncated 9-byte header, short payload, Length>MAX, ID==0, reserved R bit set) → try a 400 if the boundary is known, else TCP close; never rescan a binary stream. **Client** bcurl [-v] [--head] host:port/path: **exactly ONE Socket per invocation — never a second connection**; sends REQUEST id=1 with host, user-agent:bcurl/1.0, accept:*/, connection:keep-alive; reads until the same-ID frame with END_STREAM=1. DATA payloads go to **stdout raw** (binary-safe); -v hexdumps every frame (both directions) to **stderr**. Exit **0** on 2xx/3xx, **1** on 4xx/5xx, **2** on transport/usage error. **Pairs**: one server, one client, and the only thing that crosses between you is this spec — a client that only works against its own server is an implementation, not a protocol.

Worked example — GET /hello.txt → 200 (18 B). Full byte annotation in HEXDUMP.md.

```
REQ hdr 00 00 40 01 03 00 00 00 01 (Len 64, REQUEST, END_STREAM|END_HEADERS, ID 1)
REQ pay 01 | 00 0A /hello.txt | 00 04 | 07+host 08+bcurl/1.0 09+/* 0A+keep-alive
RESP hdr 00 00 51 02 00 00 00 01 (Len 81, RESPONSE, +DATA follows, ID 1)
RESP pay 00 C8 (200) | 00 04 | 03+ctype 04+"18" 05+date 06+bserve/1.0
DATA hdr 00 00 12 03 01 00 00 01 (Len 18, DATA, END_STREAM, ID 1)
DATA pay 68 65 6C 6C 6F 20 66 72 6F 6D 20 62 73 65 72 76 65 0A (hello from bserve\n → stdout)
```

If you cannot annotate your own bytes, the spec is not finished. This document is normative; when plan and code disagree, this spec wins. Two pages — enough for a stranger to reimplement.

BHTTP/1.0 spec · hand-in item 1 of 3 (spec + program + annotated hexdump) · big-endian · single persistent TCP connection.